

Day 5 - Topic 6: REST API Serving, Prompt Engineering & Token Limits

Week 1 - Local Inferencing Foundations

Goal: Build a minimal, production-shaped inference API from scratch using FastAPI. Add Pydantic request validation, streaming support, and load-test it. Master the system/user/assistant message structure, few-shot prompting, temperature vs top-p, and handle context windows and token limits safely.

PART A - REST API Serving with FastAPI

Why FastAPI for LLM Serving?

FastAPI is the right tool for wrapping an LLM inference engine because it provides async-first request handling (critical for non-blocking streaming), automatic OpenAPI documentation, native Pydantic validation, and near-zero boilerplate. The architecture principle here is: **keep the serving layer thin**. FastAPI handles HTTP, Pydantic handles validation, the model handles inference - business logic does not belong in any of these layers.

Production-Shaped Server Architecture

Layer	Responsibility	Tool
HTTP Server	Accept connections, route requests	uvicorn (ASGI)
API Framework	Parse JSON, validate, route handlers	FastAPI
Validation	Schema enforcement, type coercion	Pydantic v2
Inference	Tokenize, generate, decode	HuggingFace / Ollama
Streaming	Flush tokens as SSE chunks	StreamingResponse
Health	Liveness and readiness probes	/health, /ready
Observability	Request logging, metrics, tracing	structlog / prometheus

Installation

```
pip install fastapi uvicorn[standard] pydantic httpx
pip install transformers torch accelerate
# For load testing (Part A.6)
pip install httpx pytest-asyncio
```

A.1 Minimal FastAPI Inference Server

```
# server.py - minimal production-shaped LLM inference API
```

[illegible]

[illegible]

```

        max_new_tokens=req.max_new_tokens,
        streamer=streamer,
        do_sample=req.do_sample,
        temperature=req.temperature,
        top_p=req.top_p,
        pad_token_id=tokenizer.eos_token_id,
    ))
    thread.start()

    for token in streamer:
        # SSE format: data: {...}

        yield f"data: {json.dumps({'token': token})}"

    "

    yield "data: [DONE]"

    "

    thread.join()

```

A.2 Running the Server

```

# Start the server (development mode - auto-reload on file changes)
uvicorn server:app --host 0.0.0.0 --port 8080 --reload

# Production mode - multiple workers (use only for CPU; GPU needs 1 worker)
uvicorn server:app --host 0.0.0.0 --port 8080 --workers 1

# With env var override
MODEL_ID=microsoft/phi-2 uvicorn server:app --port 8080

# View auto-generated API docs
open http://localhost:8080/docs          # Swagger UI
open http://localhost:8080/redoc        # ReDoc

```

A.3 Test with curl and httpx

```

# Health check
curl http://localhost:8080/health

# Non-streaming generation
curl -s http://localhost:8080/generate \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": "The key insight about transformer models is",
    "max_new_tokens": 60,
    "temperature": 0.7
  }' | python -m json.tool

# Streaming - watch tokens arrive in real time
curl -sN http://localhost:8080/generate \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": "Explain attention in one paragraph",

```

```

        "max_new_tokens": 80,
        "stream": true
    }'

# Python httpx client (async)
import asyncio, httpx, json

async def stream_generate(prompt: str):
    async with httpx.AsyncClient(timeout=120) as client:
        async with client.stream("POST", "http://localhost:8080/generate",
                                json={"prompt": prompt, "stream": True,
                                      "max_new_tokens": 80}) as resp:
            async for line in resp.aiter_lines():
                if line.startswith("data: ") and line != "data: [DONE]":
                    data = json.loads(line[6:])
                    print(data["token"], end="", flush=True)

    print()

asyncio.run(stream_generate("The three laws of robotics are"))

```

A.4 Adding Chat (Multi-turn) Support

```

# chat_server.py - extends server.py with multi-turn chat endpoint
from pydantic import BaseModel
from typing import List, Literal

class Message(BaseModel):
    role: Literal["system", "user", "assistant"]
    content: str

class ChatRequest(BaseModel):
    messages: List[Message]
    max_new_tokens: int = 200
    temperature: float = 0.7
    top_p: float = 0.9
    stream: bool = False

def build_prompt_from_messages(messages: List[Message]) -> str:
    # Generic chat template - works for most instruction-tuned models.
    # In production, use tokenizer.apply_chat_template() for model-specific formatting.
    parts = []
    for msg in messages:
        if msg.role == "system":
            parts.append(f"[SYSTEM]\n{msg.content}\n")
        elif msg.role == "user":
            parts.append(f"[USER]\n{msg.content}\n")
        elif msg.role == "assistant":
            parts.append(f"[ASSISTANT]\n{msg.content}\n")
    parts.append("[ASSISTANT]\n") # prime the model to continue as assistant
    return "".join(parts)

@app.post("/chat")
async def chat(req: ChatRequest):
    prompt = build_prompt_from_messages(req.messages)
    # Reuse generate logic
    gen_req = GenerateRequest(

```

```

        prompt=prompt,
        max_new_tokens=req.max_new_tokens,
        temperature=req.temperature,
        top_p=req.top_p,
        stream=req.stream,
    )
    return await generate(gen_req)

# Usage:
# curl -s http://localhost:8080/chat -H "Content-Type: application/json" -d '{
#   "messages": [
#     {"role": "system", "content": "You are a concise AI assistant."},
#     {"role": "user", "content": "What is KV caching?"}
#   ],
#   "max_new_tokens": 80
# }' | python -m json.tool

```

A.5 Load Test with httpx

```

# load_test.py - concurrent load fixture using httpx
import asyncio, httpx, time, statistics

BASE_URL      = "http://localhost:8080"
CONCURRENCY   = 4          # simultaneous requests
N_REQUESTS    = 20         # total requests
PROMPT        = "Summarise the transformer architecture in two sentences."

async def single_request(client: httpx.AsyncClient, idx: int) -> float:
    t0 = time.perf_counter()
    resp = await client.post(
        f"{BASE_URL}/generate",
        json={"prompt": PROMPT, "max_new_tokens": 40, "do_sample": False},
        timeout=60,
    )
    resp.raise_for_status()
    return time.perf_counter() - t0

async def load_test():
    latencies = []
    sem       = asyncio.Semaphore(CONCURRENCY)

    async def bounded(client, idx):
        async with sem:
            lat = await single_request(client, idx)
            latencies.append(lat)
            print(f" req {idx:>3}: {lat:.2f}s")

    async with httpx.AsyncClient() as client:
        # Warm-up
        await single_request(client, 0)
        print(f"Warm-up done. Starting {N_REQUESTS} requests "
              f"(concurrency={CONCURRENCY})...\n")
        t_start = time.perf_counter()
        await asyncio.gather(*[bounded(client, i) for i in range(N_REQUESTS)])
        wall = time.perf_counter() - t_start

    p50 = statistics.median(latencies)

```

```
p95 = sorted(latencies)[int(len(latencies)*0.95)]
rps = N_REQUESTS / wall

print(f"\n{'Total requests':<20}: {N_REQUESTS}")
print(f"{'Concurrency':<20}: {CONCURRENCY}")
print(f"{'Wall time':<20}: {wall:.2f}s")
print(f"{'RPS':<20}: {rps:.2f}")
print(f"{'p50 latency':<20}: {p50:.3f}s")
print(f"{'p95 latency':<20}: {p95:.3f}s")
print(f"{'Min latency':<20}: {min(latencies):.3f}s")
print(f"{'Max latency':<20}: {max(latencies):.3f}s")

asyncio.run(load_test())
```

PART B - Prompt Engineering Basics

The System / User / Assistant Message Structure

Modern instruction-tuned LLMs are trained on conversations structured as three roles. Understanding these roles is the foundation of reliable prompt engineering:

Role	Purpose	Persists across turns?	Typical content
system	Set persona, constraints, output format	Yes - prepended always	Instructions, rules, tone, output schema
user	The human's turn - question or instruction	Per-turn	Query, task description, data to process
assistant	The model's previous reply	Per-turn (history)	Prior responses, injected few-shot examples

B.1 System / User / Assistant in Practice

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

MODEL_ID = "microsoft/phi-2" # or any instruction-tuned model
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID, trust_remote_code=True)
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID, torch_dtype=torch.float16, device_map="auto")
model.eval()

def chat(system: str, user: str, history: list = None,
        max_new_tokens: int = 200, temperature: float = 0.7) -> str:
    # Build a chat prompt and generate a response.
    # history: list of (user_msg, assistant_msg) tuples for multi-turn context.
    # Use tokenizer's built-in chat template if available
    messages = [{"role": "system", "content": system}]
    if history:
        for u, a in history:
            messages.append({"role": "user", "content": u})
            messages.append({"role": "assistant", "content": a})
    messages.append({"role": "user", "content": user})

    # apply_chat_template inserts correct special tokens for the model family
    if hasattr(tokenizer, "apply_chat_template"):
        prompt = tokenizer.apply_chat_template(
            messages, tokenize=False, add_generation_prompt=True
        )
    else:
        # Fallback: manual template
        parts = [f"[SYSTEM]\n{system}\n"]
        if history:
            for u, a in history:
                parts += [f"[USER]\n{u}\n", f"[ASSISTANT]\n{a}\n"]
        parts.append(f"[USER]\n{user}\n[ASSISTANT]\n")
        prompt = "".join(parts)

    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    with torch.no_grad():
```



```

        out = model.generate(
            **inputs, max_new_tokens=max_new_tokens,
            temperature=temperature, do_sample=temperature > 0,
            pad_token_id=tokenizer.eos_token_id,
        )
    new_ids = out[0][inputs["input_ids"].shape[1]:]
    return tokenizer.decode(new_ids, skip_special_tokens=True).strip()

# Example usage
SYSTEM = "You are a senior AI infrastructure engineer. Reply concisely and technically."

# Single turn
resp = chat(SYSTEM, "What is PagedAttention and why does it matter?")
print("=== Single Turn ===")
print(resp)

# Multi-turn with history
history = [
    ("What is PagedAttention?", resp),
]
resp2 = chat(SYSTEM, "How does it compare to standard KV cache management?", history)
print("\n=== Follow-up ===")
print(resp2)

```

B.2 Few-Shot Prompting Patterns

Few-shot prompting places worked examples (input-output pairs) in the prompt before the actual query. This steers the model's output format and style without any fine-tuning. The examples are processed as prior assistant turns.

```

# Few-shot prompting: teach the model output format via examples

SYSTEM = "You are a JSON extraction assistant. Always reply with valid JSON only."

# Few-shot examples as history (input-output pairs)
FEW_SHOT_HISTORY = [
    (
        "Extract the name and age: 'John Smith is 32 years old.'",
        '{"name": "John Smith", "age": 32}'
    ),
    (
        "Extract the name and age: 'Dr. Sarah Lee, aged 45, presented the paper.'",
        '{"name": "Sarah Lee", "age": 45}'
    ),
]

# New query - model should follow the demonstrated format
query = "Extract the name and age: 'The developer Michael Chang, 27, joined last year.'"
result = chat(SYSTEM, query, history=FEW_SHOT_HISTORY, temperature=0.0)
print(result)
# Expected: {"name": "Michael Chang", "age": 27}

# Chain-of-thought few-shot: show reasoning steps
SYSTEM_COT = "You are a reasoning assistant. Think step by step before answering."

FEW_SHOT_COT = [

```

```

        (
            "If a model has 7 billion parameters in FP16, how much VRAM does it need?",
            "Step 1: FP16 = 2 bytes per parameter.\n"
            "Step 2: 7 billion x 2 = 14 GB.\n"
            "Step 3: Add ~10% overhead for buffers.\n"
            "Answer: approximately 15.4 GB VRAM."
        ),
    ]

query_cot = "If a model has 13 billion parameters in INT4, how much VRAM does it need?"
result_cot = chat(SYSTEM_COT, query_cot, history=FEW_SHOT_COT, temperature=0.0)
print(result_cot)

```

B.3 Temperature vs Top-p - Deep Dive

Both parameters control the randomness of token sampling from the model's probability distribution, but they operate differently and interact with each other:

```

# Temperature: scales the logits before softmax
#
#   p_i = exp(logit_i / T) / sum(exp(logit_j / T))
#
# T < 1.0 -> sharpen distribution -> more deterministic, repetitive
# T = 1.0 -> unmodified distribution
# T > 1.0 -> flatten distribution -> more random, creative
# T = 0.0 -> argmax (greedy) - always picks highest probability token

# Top-p (nucleus sampling): keeps the smallest set of tokens
# whose cumulative probability >= p, then samples uniformly from that set.
#
# top_p = 0.9 -> sample from top tokens summing to 90% of probability
# top_p = 1.0 -> sample from all tokens (no filtering)
# top_p = 0.1 -> very conservative - only very likely tokens

import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

MODEL_ID = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
model = AutoModelForCausalLM.from_pretrained(MODEL_ID).eval()

prompt = "The most important quality of a software engineer is"
inputs = tokenizer(prompt, return_tensors="pt")

configs = [
    ("Greedy (T=0)", dict(do_sample=False)),
    ("Conservative (T=0.3)", dict(do_sample=True, temperature=0.3, top_p=0.9)),
    ("Balanced (T=0.7)", dict(do_sample=True, temperature=0.7, top_p=0.9)),
    ("Creative (T=1.2)", dict(do_sample=True, temperature=1.2, top_p=0.95)),
    ("Nucleus only (p=0.5)", dict(do_sample=True, temperature=1.0, top_p=0.5)),
]

print(f"Prompt: {prompt}\n")
for label, kwargs in configs:
    with torch.no_grad():
        out = model.generate(

```

```

        **inputs, max_new_tokens=25,
        pad_token_id=tokenizer.eos_token_id, **kwargs
    )
    text = tokenizer.decode(out[0][inputs["input_ids"].shape[1]:],
                           skip_special_tokens=True)
    print(f"[{label}]")
    print(f" {text.strip()}\n")

```

Use Case	Temperature	Top-p	Rationale
Code generation	0.0 – 0.2	0.95	Deterministic, syntactically correct
Factual Q&A	0.0 – 0.3	0.9	Low hallucination risk
Chatbot / assistant	0.6 – 0.8	0.9	Natural conversation, some variety
Creative writing	0.9 – 1.2	0.95	Diverse, imaginative output
Brainstorming	1.0 – 1.5	1.0	Maximum diversity, explore space
Data extraction (JSON)	0.0	1.0	Deterministic structured output

PART C - Context Windows & Token Limits

What is a Context Window?

The context window (also called max sequence length or `n_ctx`) is the maximum number of tokens the model can process in a single forward pass - both input (prompt) and output (generation) combined. It is a hard architectural limit baked into the position embeddings at training time. Exceeding it causes silent truncation or an out-of-memory crash. Modern models range from 4K (GPT-2) to 128K+ (LLaMA-3).

Model	Context Window	Notes
GPT-2	1,024 tokens	Original - very limited
LLaMA-2 7B	4,096 tokens	Standard for most 7B models
Mistral 7B	32,768 tokens	Sliding window attention
LLaMA-3 8B	8,192 tokens	Default; 128K via RoPE extension
LLaMA-3.1 8B	131,072 tokens	Official 128K context
Phi-3 Mini	4,096 / 128K	Two variants
Claude 3 Sonnet	200,000 tokens	API - largest available

Tokenization - The Core Concept

Tokens are not words. Modern LLMs use byte-pair encoding (BPE) or SentencePiece tokenization, splitting text into subword units. The ratio of tokens to words is roughly **1 word $\approx$ 1.3 tokens** for English prose, but varies widely: code has denser tokens, non-Latin scripts (Chinese, Arabic) can be 3-5 tokens/char, and numbers are tokenized digit-by-digit in some tokenizers.

```
from transformers import AutoTokenizer

# Load tokenizers for different model families
tokenizers = {
    "GPT-2 (BPE)": AutoTokenizer.from_pretrained("gpt2"),
    "LLaMA-3 (tiktoken)": AutoTokenizer.from_pretrained(
        "meta-llama/Meta-Llama-3-8B",
        token="hf_YOUR_TOKEN"), # requires HF login
}

texts = [
    "Hello, world!",
    "The KV cache stores key and value tensors for all previous tokens.",
    "def fibonacci(n): return n if n <= 1 else fibonacci(n-1) + fibonacci(n-2)",
    "1234567890",
    "AI is transforming every industry.",
]

for tok_name, tok in tokenizers.items():
    print(f"\n=== {tok_name} ===")
    for text in texts:
        ids = tok.encode(text)
```

```

tokens = tok.convert_ids_to_tokens(ids)
ratio = len(ids) / len(text.split())
print(f"  '{text[:45]}'")
print(f"    Tokens: {len(ids)}  Words: {len(text.split())}  "
      f"Ratio: {ratio:.2f}  IDs: {ids[:8]}...")

```

C.1 Counting Tokens Before Sending

```

from transformers import AutoTokenizer

MODEL_ID = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
MAX_CTX = 1024  # GPT-2's context window

def count_tokens(text: str) -> int:
    return len(tokenizer.encode(text))

def check_fits(prompt: str, max_new_tokens: int = 200) -> dict:
    prompt_tokens = count_tokens(prompt)
    total_needed = prompt_tokens + max_new_tokens
    fits = total_needed <= MAX_CTX
    headroom = MAX_CTX - total_needed

    return {
        "prompt_tokens" : prompt_tokens,
        "max_new_tokens" : max_new_tokens,
        "total_needed" : total_needed,
        "context_window" : MAX_CTX,
        "fits" : fits,
        "headroom_tokens": headroom,
    }

# Test various prompt lengths
prompts = [
    ("Short", "What is KV caching?"),
    ("Medium", "Explain the transformer architecture. " * 10),
    ("Long", "Describe each component of LLM serving in detail. " * 30),
]
for label, p in prompts:
    info = check_fits(p)
    status = "OK" if info["fits"] else "OVERFLOW"
    print(f"{label:<8} [{status}] "
          f"prompt={info['prompt_tokens']:>5} tok "
          f"total={info['total_needed']:>5} "
          f"headroom={info['headroom_tokens']:>6}")

```

C.2 Truncation Strategies

```

from transformers import AutoTokenizer
import textwrap

MODEL_ID = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
MAX_CTX = 1024

def truncate_prompt(prompt: str, max_new_tokens: int = 200,

```

```

        strategy: str = "tail") -> tuple:
# Truncate prompt to fit within context window.
# # strategy options:
# "tail" - keep the END of the prompt (default - preserves recency)
# "head" - keep the START (preserves instructions / system prompt)
# "middle" - keep start + end, drop middle (sliding window style)
max_prompt_tokens = MAX_CTX - max_new_tokens
ids = tokenizer.encode(prompt)

if len(ids) <= max_prompt_tokens:
    return prompt, len(ids), False # no truncation needed

if strategy == "tail":
    truncated_ids = ids[-max_prompt_tokens:]
elif strategy == "head":
    truncated_ids = ids[:max_prompt_tokens]
elif strategy == "middle":
    keep_each = max_prompt_tokens // 2
    truncated_ids = ids[:keep_each] + ids[-keep_each:]
else:
    truncated_ids = ids[:max_prompt_tokens]

truncated_text = tokenizer.decode(truncated_ids, skip_special_tokens=True)
return truncated_text, len(truncated_ids), True

# Demonstration
long_prompt = "Context: " + ("This is important background information. " * 100)
long_prompt += "\nQuestion: Summarise the above."

for strat in ["head", "tail", "middle"]:
    text, n_tok, was_truncated = truncate_prompt(long_prompt, strategy=strat)
    print(f"Strategy={strat:<8} tokens={n_tok:>5} truncated={was_truncated}")
    print(f" Start: {text[:60]}...")
    print(f" End : ...{text[-60:]}\\n")

```

C.3 Sliding Window for Long Documents

```

from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

MODEL_ID = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
model = AutoModelForCausalLM.from_pretrained(MODEL_ID).eval()
MAX_CTX = 1024

def sliding_window_summarize(
    document: str,
    window_tokens: int = 700,
    overlap_tokens: int = 100,
    max_new_per_chunk: int = 150,
) -> list:
    # Process a document longer than the context window using sliding windows.
    # Each chunk overlaps with the previous. Returns list of chunk summaries.
    ids = tokenizer.encode(document)
    summaries = []
    step = window_tokens - overlap_tokens

```

```

i        = 0

while i < len(ids):
    chunk_ids = ids[i : i + window_tokens]
    chunk_text = tokenizer.decode(chunk_ids, skip_special_tokens=True)
    prompt     = f"Summarise this passage in 2 sentences:\n\n{chunk_text}\n\nSummary:"

    inputs = tokenizer(prompt, return_tensors="pt",
                        max_length=MAX_CTX - max_new_per_chunk,
                        truncation=True)
    with torch.no_grad():
        out = model.generate(
            **inputs, max_new_tokens=max_new_per_chunk,
            do_sample=False, pad_token_id=tokenizer.eos_token_id,
        )
    new_ids = out[0][inputs["input_ids"].shape[1]:]
    summary = tokenizer.decode(new_ids, skip_special_tokens=True).strip()
    summaries.append({"chunk": len(summaries)+1, "summary": summary})
    print(f"  Chunk {len(summaries)}: {len(chunk_ids)} tokens processed")
    i += step

return summaries

# Usage
long_doc = "Distributed systems face fundamental challenges. " * 80
results = sliding_window_summarize(long_doc)
for r in results:
    print(f"Chunk {r['chunk']}: {r['summary'][:100]}...")

```

Testing & Metrics

API Verification Checklist

```

# Run all checks against your running server

BASE = "http://localhost:8080"

import requests, json

checks = []

# 1. Health check
r = requests.get(f"{BASE}/health")
checks.append(("GET /health == 200", r.status_code == 200))
checks.append(("model_loaded == true", r.json().get("model_loaded") == True))

# 2. Basic generation
r = requests.post(f"{BASE}/generate",
                  json={"prompt": "Hello", "max_new_tokens": 10})
checks.append(("POST /generate == 200", r.status_code == 200))
checks.append(("response has 'text' field", "text" in r.json()))
checks.append(("input_tokens > 0", r.json().get("input_tokens", 0) > 0))
checks.append(("output_tokens > 0", r.json().get("output_tokens", 0) > 0))

# 3. Pydantic validation - should return 422 on bad input

```

```

r_bad = requests.post(f"{BASE}/generate",
    json={"prompt": "", "max_new_tokens": 99999})
checks.append(("Empty prompt returns 422", r_bad.status_code == 422))

r_bad2 = requests.post(f"{BASE}/generate",
    json={"temperature": 5.0, "prompt": "test"})
checks.append(("temperature>2.0 returns 422", r_bad2.status_code == 422))

# 4. Streaming returns SSE
r_stream = requests.post(f"{BASE}/generate",
    json={"prompt": "Say hi", "stream": True, "max_new_tokens": 10}, stream=True)
first_line = next(r_stream.iter_lines()).decode()
checks.append(("stream starts with 'data:', first_line.startswith('data:'))))

# Print results
print(f"\n{'Check':<45} {'Result':>8}")
print("-" * 55)
for desc, passed in checks:
    mark = "PASS" if passed else "FAIL"
    print(f"{desc:<45} {mark:>8}")
total = sum(1 for _, p in checks if p)
print(f"\n{total}/{len(checks)} checks passed")

```

Metrics Summary

Metric	Measurement method	Target
Server startup time	time uvicorn server:app	< 15s (model load)
p50 latency	load_test.py statistics.median	< 5s (CPU 7B, 100 tok)
p95 latency	sorted(latencies)[int(n*0.95)]	< 2x p50
RPS (throughput)	n_requests / wall_time	>1 RPS single GPU
Validation reject	curl bad input -> 422	Always 422, never 500
Streaming TTFT	Timer in stream client	< 3s CPU, < 1s GPU
Token count accuracy	input_tokens matches tokenizer.encode	Exact match

Key Takeaways

- **Keep FastAPI thin** - it handles HTTP and validation only. Model loading, batching, and generation logic must be decoupled from route handlers.
- **Always use Pydantic** to validate inputs with explicit min/max bounds before they reach the model. An unchecked max_new_tokens=100000 will OOM your server.
- **TextIteratorStreamer in a background thread** is the correct pattern for non-blocking SSE streaming in FastAPI - the main thread iterates the generator.
- **apply_chat_template()** is the correct way to format messages - it uses model-specific special tokens rather than a generic string template.
- **Temperature=0 + top_p=1** for structured/JSON output. Temperature=0.7 + top_p=0.9 for balanced conversational tasks.

- **Always count tokens before generation** - check prompt + max_new_tokens fits within the model's context window or you will get truncated/OOM errors.
- **Sliding window** is the practical strategy for documents longer than the context window - overlap chunks by ~100 tokens to preserve continuity.

Week 1 is now complete through Day 5. Next topic: **Day 6 - Containers and Packaging**: write a Dockerfile for your FastAPI server, use multi-stage builds, mount model weights as volumes, and add /health and /version endpoints for production deployment.